

# Formalizing the Markov de Finetti Theorem in Lean 4: Current Status and Open Gap

Mettapedia project technical note

April 2026

## Abstract

We report on an ongoing Lean 4 formalization of the Markov de Finetti theorem (Diaconis–Freedman 1980): every Markov-exchangeable, recurrent prefix measure on a finite alphabet is a mixture of time-homogeneous Markov chains.

The formalization is a partial success with a precisely characterized gap. The *internal* representation theorem—given an extension to infinite trajectories together with explicit row-kernel data, construct the mixing measure—is fully proved (zero `sorry`). The carrier transport bijection—an evidence-preserving Equiv between finite prefix carriers for adjacent visit indices—is fully proved (1465 lines, zero `sorry`). The *literature-facing* theorem, which derives the row-kernel data from bare Markov exchangeability and recurrence, requires a bridge from the proved carrier transport to the proved de Finetti infrastructure.

A structural audit (April 2026) identified three bugs in the gap characterization: (1) the evidence-preserving Equiv is used far below its full strength (only 1D marginals extracted when it preserves all transition counts); (2) the zero-sorry count masks unproved mathematical content encoded as assumption interfaces; (3) an archived Euler trail infrastructure (1225 lines, 0 sorry) provides a more direct route to the missing bridge. The corrected proof strategy (§5) estimates ~530 lines of new code to close the gap entirely, primarily a bridge from evidence preservation to successor-matrix partial exchangeability.

The formalization comprises approximately 50,000 lines of Lean across 12 active files, with an additional 43,000 lines formalizing the classical (i.i.d.) de Finetti theorem via three independent proof strategies. All active code compiles with zero `sorry`.

## Contents

|          |                                                                                |          |
|----------|--------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction and Motivation</b>                                             | <b>2</b> |
| <b>2</b> | <b>Mathematical Setup</b>                                                      | <b>3</b> |
| 2.1      | Prefix measures . . . . .                                                      | 3        |
| 2.2      | Transition counts and Markov evidence . . . . .                                | 3        |
| 2.3      | Markov exchangeability . . . . .                                               | 3        |
| 2.4      | Markov parameter space and word probability . . . . .                          | 4        |
| 2.5      | Recurrence . . . . .                                                           | 4        |
| <b>3</b> | <b>What Is Proved</b>                                                          | <b>5</b> |
| 3.1      | Dirichlet conjugacy and structured predictors . . . . .                        | 5        |
| 3.2      | Domain restriction: Markov exchangeability implies count sufficiency . . . . . | 6        |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 3.3      | Visit-time infrastructure (hard direction)                               | 6         |
| 3.4      | The internal representation theorem                                      | 7         |
| <b>4</b> | <b>The Gap: Refined Analysis</b>                                         | <b>9</b>  |
| 4.1      | Architecture overview                                                    | 9         |
| 4.2      | What is already derived from PE                                          | 10        |
| 4.3      | What is NOT yet derived: the start-restricted bottleneck                 | 10        |
| 4.4      | The carrier transport argument                                           | 11        |
| 4.5      | The cross-row coherence gap (overlooked until April 2026)                | 12        |
| 4.6      | Note on the recurrence definition                                        | 13        |
| 4.7      | Archived approaches and reusable infrastructure                          | 13        |
| 4.8      | Note on the non-derivability theorem                                     | 13        |
| 4.9      | Summary: the refined gap                                                 | 14        |
| <b>5</b> | <b>Proof Strategy: Structural Audit and Path to Completion</b>           | <b>15</b> |
| 5.1      | Bug 1: The evidence-preserving Equiv is used far below its full strength | 15        |
| 5.2      | Bug 2: Zero-sorry masking of unproved mathematical content               | 16        |
| 5.3      | Bug 3: The Euler trail archive is the missing bridge                     | 17        |
| 5.4      | Corrected proof strategy                                                 | 17        |
| 5.5      | Remaining mathematical gap: the wiring step                              | 18        |
| 5.6      | Summary of recommended steps                                             | 19        |
| <b>6</b> | <b>The Broader Context</b>                                               | <b>19</b> |
| 6.1      | Classical de Finetti (i.i.d. case)                                       | 19        |
| 6.2      | Solomonoff bridge: exchangeability implies count sufficiency             | 19        |
| 6.3      | The complete chain (with gap)                                            | 20        |
| <b>7</b> | <b>File Inventory</b>                                                    | <b>20</b> |

# 1 Introduction and Motivation

Solomonoff induction (1964) provides a universal Bayesian mixture over all computable hypotheses. In practice, one restricts to structured hypothesis classes—i.i.d. models, finite-state Markov models, hidden Markov models—for tractability and interpretability. The conceptual bridge is:

*Exchangeability (or partial exchangeability) assumptions identify when a rich family of distributions can be represented as a mixture over a simpler parametric family.*

For i.i.d. sequences, de Finetti’s theorem (1931/1937) gives exchangeability  $\Rightarrow$  mixture of i.i.d. laws. For Markov sequences, Diaconis and Freedman (1980) show a corresponding representation for *Markov-exchangeability*, with an additional *recurrence* hypothesis.

Once such structure theorems are formalized, they justify *structured universal mixtures*: Solomonoff-like prediction instantiated by mixing over the parametric space of Markov models, with hyperpriors retaining dominance-style guarantees. This is the broader program that motivates the formalization.

**Status summary.** The internal representation theorem is proved: given an extension measure  $P$  on infinite trajectories together with explicit row-kernel crux data, we construct the mixing measure  $\pi$  on the Markov parameter space and verify  $\mu(xs) = \int \text{wordProb}_\theta(xs) d\pi(\theta)$  for all finite words  $xs$ .

The literature-facing theorem—deriving the row-kernel data from bare Markov exchangeability and recurrence—requires three sub-obligations (Section 4) that remain open. A structural audit (§4.5) identified a previously overlooked gap in the cross-row coherence argument.

## 2 Mathematical Setup

Throughout,  $k \geq 1$  is a fixed positive integer and the state space is  $A = \text{Fin}(k)$ .

### 2.1 Prefix measures

A *prefix measure* assigns a nonnegative extended real to each finite word over  $A$ , satisfying cylinder consistency:  $\mu(\varepsilon) = 1$  and  $\mu(xs) = \sum_{a \in A} \mu(xs \cdot [a])$ . In the Lean development, this is modeled as:

```
-- FiniteAlphabet.PrefixMeasure (Fin k)
-- A function List (Fin k) → ENNReal satisfying consistency
```

### 2.2 Transition counts and Markov evidence

For a trajectory  $x_0, x_1, \dots, x_n$  over  $\text{Fin}(k)$ , the Markov sufficient statistics are the starting state  $x_0$  and the transition-count matrix  $N(i, j) = \#\{t < n : x_t = i, x_{t+1} = j\}$ .

```
def transCount (xs : Fin (n + 1) → α) (a b : α) : ℕ :=
  (univ.filter (fun i : Fin n =>
    xs (Fin.castSucc i) = a ∧ xs (Fin.succ i) = b)).card

structure MarkovEvidence (α : Type*) where
  start : α
  trans : α → α → ℕ

def evidenceOf (xs : Fin (n + 1) → α) : MarkovEvidence α :=
  { start := xs 0
    trans := fun a b => transCount xs a b }
```

### 2.3 Markov exchangeability

A prefix measure is *Markov-exchangeable* when  $\mu(xs)$  depends only on the Markov evidence, not on the detailed order of transitions:

```
def FiniteMarkovExchangeable (n : ℕ)
  (X : Fin (n + 1) → → α) (μ : Measure) : Prop :=
  ∀ vals₁ vals₂ : Fin (n + 1) → α,
    evidenceOf vals₁ = evidenceOf vals₂ →
      μ (cylinder X vals₁) = μ (cylinder X vals₂)
```

For prefix measures (rather than random variables), the corresponding predicate is:

```

def MarkovExchangeablePrefixMeasure
  (μ : PrefixMeasure (Fin k)) : Prop :=
  ∀ xs ys : List (Fin k),
    -- if xs and ys have the same length and same evidence:
    (∃ (n : ℕ), xs = List.ofFn xs_fin ∧
      ys = List.ofFn ys_fin ∧
      evidenceOf xs_fin = evidenceOf ys_fin) →
      μ xs = μ ys

```

**Example 2.1** (Worked example:  $k = 2$ , trajectory of length 4). Let  $A = \{0, 1\}$  and consider the trajectory  $xs = [0, 1, 0, 1]$  of length 4 (so  $n = 3$ ). The Markov evidence is:

$$\text{start} = 0, \quad N = \begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix},$$

recording two  $0 \rightarrow 1$  transitions and one  $1 \rightarrow 0$  transition. The trajectory  $[0, 1, 1, 0]$  has evidence  $\text{start} = 0$ ,  $N = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$ , which differs (one fewer  $0 \rightarrow 1$ , one more  $1 \rightarrow 1$ ). A Markov-exchangeable measure may assign different probabilities to these.

However,  $[0, 1, 0, 1]$  and any other length-4 trajectory starting at 0 with the same count matrix  $N$  must receive the same probability. In this case, no other trajectory has exactly this count matrix, so the equivalence class is a singleton.

The parameter space  $\Theta_2 = \text{MarkovParam}(2)$  is  $\Delta_1 \times \Delta_1 \times \Delta_1$ : an initial distribution on  $\{0, 1\}$  (a single parameter  $p_0 = \mathbb{P}(X_0 = 0)$ ), and two row-stochastic transition probabilities  $p_{0 \rightarrow 1}$  and  $p_{1 \rightarrow 0}$ . For  $\theta = (p_0, p_{0 \rightarrow 1}, p_{1 \rightarrow 0})$ :

$$\text{wordProb}_\theta([0, 1, 0, 1]) = p_0 \cdot p_{0 \rightarrow 1} \cdot p_{1 \rightarrow 0} \cdot p_{0 \rightarrow 1} = p_0 \cdot p_{0 \rightarrow 1}^2 \cdot p_{1 \rightarrow 0}.$$

The representation theorem, when applicable, gives  $\mu([0, 1, 0, 1]) = \int_{\Theta_2} p_0 \cdot p_{0 \rightarrow 1}^2 \cdot p_{1 \rightarrow 0} d\pi(\theta)$  for some probability measure  $\pi$  on  $\Theta_2$ .

## 2.4 Markov parameter space and word probability

A Markov model on  $\text{Fin}(k)$  is parameterized by an initial distribution  $\pi_0$  and a row-stochastic transition kernel  $P(i, \cdot)$ :

$$\text{wordProb}_\theta(x_0, \dots, x_n) = \pi_0(x_0) \cdot \prod_{t=0}^{n-1} P(x_t, x_{t+1}).$$

The parameter space  $\Theta_k = \text{MarkovParam}(k)$  is a finite product of simplices, hence compact. The function  $\theta \mapsto \text{wordProb}_\theta(xs)$  is continuous on  $\Theta_k$ .

## 2.5 Recurrence

Markov exchangeability alone does *not* imply a mixture representation. A counterexample is constructed in the Lean development: consider  $k = 2$  with the deterministic chain  $0 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow \dots$  (the chain leaves state 0 at step 1 and never returns). This prefix measure is Markov-exchangeable—the only trajectory of any given length starting at 0 is the all-ones tail, so the “permutation” equivalence class is always a singleton, and the condition holds vacuously. But

it cannot be represented as a mixture of time-homogeneous Markov chains, because any non-degenerate mixture would assign positive probability to trajectories that return to 0, contradicting the deterministic behavior.

The Diaconis–Freedman recurrence condition requires the trajectory to return to its starting state infinitely often, almost surely:

```
def recurrentEvent : Set (ℕ → Fin k) :=
  N : ℕ,    n ≥ N, returnEvent n

def MarkovRecurrentPrefixMeasure
  (μ : PrefixMeasure (Fin k)) : Prop :=
  ∃ (P : Measure (ℕ → Fin k)),
    IsProbabilityMeasure P ∧
    (∀ xs : List (Fin k),
      μ xs = P (cylinder xs)) ∧
    P (recurrentEvent) = 1
```

Recurrence is an *identifiability anchor*: it guarantees the trajectory keeps resampling transitions, allowing long-run statistics to stabilize.

### 3 What Is Proved

All results in this section compile with zero `sorry` in Lean 4 with Mathlib.

#### 3.1 Dirichlet conjugacy and structured predictors

The Dirichlet-multinomial conjugate prior pattern is formalized for  $k$ -ary alphabets. Evidence is a vector of counts:

```
structure MultiEvidence (k : ℕ) where
  counts : Fin k → ℕ

structure DirichletParams (k : ℕ) where
  priorParams : Fin k → ℝ
  params_pos : ∀ i, 0 < priorParams i
```

The key theorem states that Bayesian update equals coordinatewise count addition—the conjugacy that makes finite-dimensional Markov inference tractable:

```
-- Verbatim from EvidenceDirichlet.lean:
theorem evidence_aggregation_is_dirichlet_update
  (prior : DirichletParams k)
  (e₁ e₂ : MultiEvidence k) (i : Fin k) :
  (⟨prior, e₁ + e₂⟩ :
    EvidenceDirichletParams k).posteriorParam i =
  (⟨prior, e₁⟩ :
    EvidenceDirichletParams k).posteriorParam i
  + e₂.counts i
```

Here `EvidenceDirichletParams` bundles a prior with observed evidence, and `posteriorParam i` returns  $\alpha_i + n_i$  (the  $i$ -th Dirichlet posterior parameter). The theorem says combining evidence

and then computing the posterior is the same as computing the posterior of the first batch and adding the second batch’s counts—the Dirichlet conjugate update pattern.

Building on this, a finite-alphabet Markov(1) predictor with one Dirichlet prior per transition row is constructed (`MarkovDirichletPredictor.lean`, 497 lines).

A *universal hyperprior mixture* over a countable family of symmetric Dirichlet priors (parameter  $a \in \{1/2, 1, 3/2, \dots\}$ , including the Jeffreys and Laplace choices) is then constructed with computable weights and dominance guarantees (`MarkovDirichletHyperpriorMixture.lean`).

### 3.2 Domain restriction: Markov exchangeability implies count sufficiency

The bridge file (`MarkovExchangeabilityBridge.lean`, 600+ lines) establishes that Markov exchangeability of a prefix measure implies predictions factor through transition-count evidence. The key constructions are:

```
-- Extract the transition-count summary from a trajectory:
def countsOfFn (xs : Fin (n + 1) → Fin k) :
  TransCounts k :=
  ⟨fun a b => transCount xs a b⟩

-- Verify the summary agrees with the evidence:
theorem summary_ofFn (xs : Fin (n + 1) → Fin k) :
  TransCounts.summary (List.ofFn xs) =
    some (countsOfFn xs, xs (Fin.last n))
```

This is the Markov analogue of the i.i.d. count-sufficiency result: under Markov exchangeability, two trajectories with the same start state and transition counts receive the same probability, regardless of the order in which transitions occur.

### 3.3 Visit-time infrastructure (hard direction)

The hard direction of the Markov de Finetti theorem requires decomposing trajectories by *visit times* to each state. The Core file (`MarkovDeFinettiFortiniBridgeCore.lean`, 1843 lines) formalizes:

```
def visitCountBefore (ω : ℕ → Fin k) (i : Fin k)
  (t : ℕ) : ℕ :=
  Finset.sum (Finset.range t)
    (fun s => if ω s = i then 1 else 0)

def isNthVisitTime (ω : ℕ → Fin k) (i : Fin k)
  (n t : ℕ) : Prop :=
  ω t = i ∧ visitCountBefore ω i t = n

noncomputable def rowSuccessorAtNthVisit
  (i : Fin k) (n : ℕ) (ω : ℕ → Fin k) : Fin k :=
  match nthVisitTime ω i n with
  | some t => successorAt ω t
  | none => i
```

For each state  $i$ , the *row-successor process*  $n \mapsto \text{rowSuccessorAtNthVisit}(i, n, \omega)$  records the state visited *after* the  $n$ -th visit to  $i$ . Under Markov exchangeability, this process has strong symmetry properties.

### 3.4 The internal representation theorem

The central achievement of the formalization is the *internal* representation theorem. It requires as input:

1. An extension  $P$  (a probability measure on  $\mathbb{N} \rightarrow \text{Fin}(k)$ ) whose cylinders agree with  $\mu$ .
2. A family of row kernels  $\text{rowKernel} : \text{Fin}(k) \rightarrow (\mathbb{N} \rightarrow \text{Fin}(k)) \rightarrow \text{ProbabilityMeasure}(\text{Fin}(k))$ .
3. *Row-kernel crux data*: measurability conditions, start-restricted invariance, and a cross-row coherence step.

**Definition 3.1** (Cross-Anchor Product Identity). The cross-anchor product identity asserts that cylinder probabilities under  $P$  decompose as integrals of row-kernel step products, conditioned on the starting state:

```
def CrossAnchorProductIdentity
  (P : Measure (N → Fin k))
  (rowKernel : Fin k → (N → Fin k)
    → ProbabilityMeasure (Fin k)) : Prop :=
  ∀ (a b : Fin k) (xs : List (Fin k)),
    P (cylinder (a :: b :: xs)) =
      ∫- ω, (if ω0 = a
        then rowKernelStepProd rowKernel ω
          (a :: b :: xs)
        else 0) ∂P
```

**Definition 3.2** (Row-Kernel Crux Data). The crux data bundles four conditions on a row-kernel family, each ensuring a measurability or coherence property needed by the internal theorem:

```
def RowKernelCruxData
  (P : Measure (N → Fin k))
  (rowKernel : Fin k → (N → Fin k)
    → ProbabilityMeasure (Fin k)) : Prop :=
  -- (1) Pointwise measurability: for each state
  --   i and target b, the map r ↦ rowKernel(i,r)({b})
  --   is AEMeasurable w.r.t. the row-process law.
  (∀ i b : Fin k, AEMeasurable
    (fun r => (rowKernel i r : Measure (Fin k))
      ({b} : Set (Fin k))))
  (rowProcessLaw P i) ∧
  -- (2) Start-restricted invariance: the kernel
  --   reproduces cylinder probabilities when
  --   restricted to trajectories starting at a
  --   given state.
  StartRestrictedRowKernelData P rowKernel ∧
  -- (3) Product measurability: the product-measure
  --   construction is AEMeasurable (needed for
  --   Fubini-style arguments).
  (∀ i : Fin k, AEMeasurable
    (fun r => Measure.pi
      (fun _ : Fin 1 => (rowKernel i r :
        Measure (Fin k)))))
```

```

    (rowProcessLaw P i)) ∧
  -- (4) Cross-row coherence: if the product identity
  --   holds for (b::c::xs), it extends by one step
  --   to (a::b::c::xs).
  CrossRowCoherenceStep P rowKernel

```

Conditions (1) and (3) are measurability obligations. Condition (2) ensures the kernel is consistent with cylinder probabilities on each start-state fiber. Condition (4) is the inductive step that extends the cross-anchor product identity from length-2 prefixes to arbitrary lengths.

The internal theorem and its proof:

```

def FortiniSuccessorMatrixInvarianceTheoremInternal
  (k : ℕ) : Prop :=
  ∀ μ : PrefixMeasure (Fin k),
  (∃ (P : Measure (ℕ → Fin k)),
    IsProbabilityMeasure P ∧
    (∀ xs, μxs = P (cylinder xs)) ∧
    ∃ rowKernel, RowKernelCruxData P rowKernel) →
  ∃ (pi : Measure (MarkovParam k)),
    IsProbabilityMeasure pi ∧
    ∀ xs, μxs = ∫- θ, wordProb θxs ∂pi

```

**Theorem 3.3** (Internal Representation, [proved, 0 sorry](#)). *fortiniSuccessorMatrixInvarianceTheoremInternal*

Given an extension  $P$  with row-kernel crux data, the mixing measure  $\pi$  on  $\text{MarkovParam}(k)$  is constructed as the pushforward of  $P$  through the map  $\omega \mapsto \text{rowKernelToMarkovParam}(\omega)$ .

```

-- Full proof (verbatim from source):
theorem fortiniSuccessorMatrixInvarianceTheoremInternal_proved :
  FortiniSuccessorMatrixInvarianceTheoremInternal k :=by
  intro μ⟨P, hPprob, hExt, rowKernel, hcrux⟩
  rcases hcrux with ⟨hEval, hrow_restrict_data, hPi, hstep⟩
  letI : IsProbabilityMeasure P :=hPprob
  have hθ :=aemeasurable_rowKernelToMarkovParam_diracInit_lifted
    P rowKernel hEval
  have hCM :=cylinderMixingIdentity_P_of_rowKernelData
    P rowKernel ⟨hEval, hrow_restrict_data, hPi, hstep⟩
  have hall :=
    rowKernelToMarkovParamLaw_reconstruction_all_diracInit_of_lifted_rowKernel
    P rowKernel hθ hCM
  set law :=rowKernelToMarkovParamLaw P
  (initKernel :=fun ω=>
    ⟨Measure.dirac (ω 0),
     Measure.dirac.isProbabilityMeasure⟩)
  (liftedRowKernelFromRowProcess rowKernel)
  have hl原因_prob : IsProbabilityMeasure law :=
    Measure.isProbabilityMeasure_map
    (f := rowKernelToMarkovParam
     (initKernel :=fun ω=>
       ⟨Measure.dirac (ω 0),
        Measure.dirac.isProbabilityMeasure⟩)
     (liftedRowKernelFromRowProcess rowKernel)) hθ

```



```

refine ⟨law, hlaw_prob, ?_⟩
intro xs
rw [hExt xs]
exact hall xs

```

The proof strategy is:

1. Verify that  $\omega \mapsto \text{rowKernelToMarkovParam}(\omega)$  is **AEMeasurable** (from the crux data).
2. Establish the *cylinder mixing identity*: for each word  $xs$  of length  $\geq 2$ ,  $P(\text{cylinder}(xs))$  equals the integral of  $\text{wordProb}$  against the pushforward law.
3. Define  $\text{law} = P \circ \text{rowKernelToMarkovParam}^{-1}$  (the pushforward measure on  $\text{MarkovParam}(k)$ ).
4. Show  $\text{law}$  is a probability measure and reconstruct  $\mu(xs) = \int \text{wordProb}_\theta(xs) d\text{law}(\theta)$ .

## 4 The Gap: Refined Analysis

The *literature-facing* theorem takes only Markov exchangeability and recurrence as hypotheses—no explicit row-kernel data:

```

abbrev FortiniCanonicalRepresentationTheorem
(k : ℕ) : Prop :=
  ∀ μ : PrefixMeasure (Fin k),
    MarkovExchangeablePrefixMeasure μ →
    MarkovRecurrentPrefixMeasure μ →
    ∃ (pi : Measure (MarkovParam k)),
      IsProbabilityMeasure pi ∧
      ∀ xs, μ xs = ∫- θ, wordProb θ xs ∂pi

```

This is *not* proved directly. The Lean source contains multiple decomposition routes, all of which reduce to a common set of sub-obligations. A recent structural audit (April 2026) clarified which obligations are essential and which arise from the decomposition architecture.

### 4.1 Architecture overview

The proof decomposes into two layers, each with sub-obligations:

**Layer 1: Extension with partial exchangeability.** From Markov exchangeability and recurrence of the prefix measure  $\mu$ , construct an extension  $P$  (a probability measure on infinite trajectories) satisfying *successor-matrix partial exchangeability*:

**Definition 4.1** (Successor-Matrix Partial Exchangeability). A measure  $P$  on  $\mathbb{N} \rightarrow \text{Fin}(k)$  is *successor-matrix partially exchangeable* when, for every finite selection of anchor states and visit indices, independent within-row permutations leave the joint law of row-successor values invariant:

```

def SuccessorMatrixPartialExchangeable
(P : Measure (ℕ → Fin k)) : Prop :=
  ∀ (m : ℕ) (anchor : Fin m → Fin k)
    (idx : Fin m → ℕ) (σ : Fin k → Equiv.Perm ℕ),
    Measure.map
      (fun ω => fun j : Fin m =>

```

```

rowSuccessorVisitProcess (anchor j) ω
  ((σ (anchor j)) (idx j))) P
=
Measure.map
  (fun ω=> fun j : Fin m =>
    rowSuccessorVisitProcess (anchor j) ω
      (idx j)) P

```

Informally: permuting visit indices independently per row does not change the joint law. This is de Finetti-style exchangeability applied per row of the transition matrix.

**Layer 2: Row-kernel construction and start-restricted coherence.** From  $P$  with successor-matrix PE, construct measurable row kernels satisfying the conditions needed by the internal theorem.

The internal theorem (Section 3.4) requires four conditions bundled as `RowKernelCruxData`. Of these, three are automatically derived from PE by existing infrastructure. The fourth—*start-restricted row-kernel factorization*—is the structural bottleneck.

## 4.2 What is already derived from PE

```

-- PROVED: PE gives row kernels with unrestricted
-- factorization + measurability
theorem exists_rowKernel_hrow_hEval_hPi_of_successorMatrixPE
  (hk : 0 < k)
  (P : Measure (N → Fin k)) [IsProbabilityMeasure P]
  (hPE : SuccessorMatrixPartialExchangeable P) :
  ∃ rowKernel,
    -- (1) Unrestricted row-process factorization
    (∀ i, ∀ (m : N) (sel : Fin m → N),
      StrictMono sel → ...) ∧
    -- (2) Singleton-eval AE-measurability
    (∀ i b, AEMeasurable ...) ∧
    -- (3) Fin 1 product-kernel AE-measurability
    (∀ i, AEMeasurable ...)

```

This is proved by applying the i.i.d. de Finetti machinery (from `external/exchangeability/`) to each row’s permutation-invariant law, then extracting the directing measure as the row kernel.

## 4.3 What is NOT yet derived: the start-restricted bottleneck

The *cross-anchor product identity*—the core of the representation— conditions on the starting state:

$$P(\text{cyl}(a :: b :: xs)) = \int_{\Omega} \mathbf{1}[\omega_0 = a] \cdot \prod_t \text{rowKernel}(x_t)(\text{rsp}_{x_t}(\omega))(\{x_{t+1}\}) \, dP(\omega).$$

The unrestricted row factorization (proved from PE) gives factorization of `rowProcessLaw P i` (all trajectories). But the product identity needs factorization of `rowProcessLaw (P.restrict {ω0 = a}) i` (trajectories conditioned on starting at  $a$ ).

**Remark 4.2** (Why start-restriction is mathematically necessary). An audit confirmed that `SuccessorMatrixPart`.`P` does *not* imply `SuccessorMatrixPartialExchangeable` (`P.restrict S`) for arbitrary measurable  $S$ . PE asserts  $P(f^{-1}(T)) = P(g^{-1}(T))$  globally, but the start-restricted version needs  $P(S \cap f^{-1}(T)) = P(S \cap g^{-1}(T))$ —a strictly stronger claim. A theorem in the Lean source (`not_derivable_startRestrictedRowSuccessorPermInvariant.of_witness`) explicitly records this non-derivability.

The existing code provides a *different* route to start-restricted properties: not from PE directly, but from Markov exchangeability of  $\mu$  via a *carrier transport* argument.

#### 4.4 The carrier transport argument

The start-restricted permutation invariance is derived (lines 1375–1620 of the Crux file) from Markov exchangeability of  $\mu$  via finite combinatorics:

1. Fix a state  $i$ , a target value  $b$ , and two visit indices  $n, n'$ .
2. Consider the set of length- $N$  trajectory prefixes (the “carrier”) consistent with each visit event.
3. Construct a bijection between the two carriers that preserves Markov evidence (start state + transition counts).
4. By Markov exchangeability, the two carriers have equal  $\mu$ -measure, hence equal  $P$ -measure on their cylinders.
5. Taking  $N \rightarrow \infty$  gives the start-restricted invariance.

```
-- The carrier: finite prefixes consistent with a
-- visit-time event, as an explicit Finset:
def rowVisitCylinderEventUpToPrefixCarrier
  (i : Fin k) (S : Finset N) (v : N → Fin k)
  (N : N) : Finset (Fin (N + 1) → Fin k) :=
  Finset.univ.filter (fun xs =>
    rowVisitCylinderEventUpTo i S v N
    (prefixExtend N xs))
```

The carrier transport assumption required by the existing theorems is:

```
-- Required: for each (i, b, n, n', N) with b ≠ i,
-- an evidence-preserving bijection between carriers
hCarrierTransport :
  ∀ (i b : Fin k) (n n' N : N), b ≠ i →
    ∃ e : carrier(i, {n}, ..., N)
      carrier(i, {n'}, ..., N),
      ∀ xs, evidenceOf xs.1 = evidenceOf (e xs).1
```

**Theorem 4.3** (Carrier Transport Bijection, *proved, 0 sorry*). For each state  $i$ , target  $b \neq i$ , adjacent visit indices  $n, n + 1$ , and horizon  $N$ , there exists an evidence-preserving bijection between the corresponding prefix carriers:

```

theorem carrierTransportEquivAdjacent
  (i b : Fin k) (n N : ℕ) (hbi : b ≠ i)
  (hSuff : ...) :
  ∃ e : carrier(i, {n}, ..., N)
    carrier(i, {n+1}, ..., N),
  ∀ xs, evidenceOf xs.1 = evidenceOf (e xs).1

```

The construction uses trajectory segment swapping (*segmentSwap*): the excursion between the  $n$ -th and  $(n+1)$ -th visits to  $i$  is transposed with the excursion between the  $(n+1)$ -th and  $(n+2)$ -th visits, preserving all transition counts. The *Equiv* is built from *rawSwap* (marked *@[reducible]* to enable kernel transparency), with *rawSwap\_selfInverse* providing the self-inverse property via internal visit-time proof construction.

The general case (arbitrary  $n \rightarrow n'$ ) follows by composing adjacent *Equivs* via *Equiv.trans* and *Nat* induction on  $|n - n'|$ . This composition step is straightforward but not yet implemented.

## 4.5 The cross-row coherence gap (overlooked until April 2026)

A structural audit in April 2026 revealed a second gap hidden by the decomposition architecture. The reduced assembly theorem (*successorMatrixPEToBuiltRowKernelOnExtension\_of\_start\_and\_invariance*, line 2112 of the *Crux* file) requires **two independent** assumptions beyond what PE provides:

1. *hStartFromPE*:  $\text{PE} \Rightarrow \text{StartRestrictedRowKernelData}$  — the start-restricted row-law factorization (addressed by carrier transport, §4.4).
2. *hInvFromPE*:  $\text{PE} \Rightarrow \text{RowSuccessorMatrixInvariance}$  — the cylinder mixing identity for all finite prefixes of length  $\geq 2$ .

Prior descriptions of the gap identified only item (1). Item (2) is a genuinely different requirement: it asserts that the product of row-kernel evaluations *across different anchor states* correctly reproduces cylinder probabilities.

**Root cause: individual vs. joint de Finetti.** The existing code applies the de Finetti theorem to each row process *independently* (line 1527 of the *Core* file):

```

have hciid :
  ∀ i : Fin k,
  Exchangeability.CconditionallyIID
    (rowProcessLaw P i)
  (fun n r => r n) :=...

```

This gives, for each anchor  $i$ , that the row- $i$  process is conditionally i.i.d. directed by *rowKernel* i. But *rowKernelStepProd* multiplies evaluations across different anchors:

$$\text{rowKernelStepProd}(\omega, [a, b, c, \dots]) = \text{rowKernel}_a(\text{rsp}_a(\omega))(\{b\}) \cdot \text{rowKernel}_b(\text{rsp}_b(\omega))(\{c\}) \cdot \dots$$

For this product to equal the cylinder probability, the row processes at different anchors must be *conditionally independent given  $\omega$* . This is the content of the *joint* de Finetti theorem for exchangeable arrays, which PE provides at the joint-law level but which the code never extracts.

**Remark 4.4** (Relationship between the two sub-gaps). If the joint de Finetti extraction is formalized (establishing cross-row conditional independence from PE), then *hInvFromPE* follows from

`hStartFromPE` by induction on prefix length: the base case is the pair-cylinder identity (from start-restricted factorization), and the induction step uses cross-row conditional independence to extend by one state. In that case, the two sub-gaps collapse into: (a) carrier transport for `hStartFromPE`, and (b) joint de Finetti extraction from PE.

## 4.6 Note on the recurrence definition

`MarkovRecurrentPrefixMeasure` requires only that the starting state  $\omega_0$  is visited infinitely often almost surely. The downstream machinery (`StrongRecurrence`, line 856 of the Crux file) needs *all* states to be visited infinitely often. The stronger `MarkovRowRecurrentPrefixMeasure` provides this.

Under Markov exchangeability on a finite state space, starting-state recurrence should imply all-state recurrence (by exchangeability, all transition rows have the same asymptotic behavior), but this implication is *not formalized*. The strong recurrence variant of the main theorem uses the stronger hypothesis, sidestepping this issue.

## 4.7 Archived approaches and reusable infrastructure

The `_archive/MarkovDeFinetti/` directory contains 37 files ( $\sim 5,000$  lines) from prior proof attempts. Notable among these:

- **Euler trail approach** (`EulerTrails`, `EulerTrailFiber`, `CopyPerm`, `BESTCore`; 1,225 lines, **0 sorry**). Trajectories with prescribed transition counts are modeled as Euler trails on a multigraph. “Copy permutations” (permuting which copy of each edge is used) preserve vertex sequences. Key proved results:
  - `trailVertexSeq_applyCopyPerm`: copy perms preserve trajectories.
  - `applyCopyPerm_isEulerTrail`: copy perms preserve the Euler trail property.
  - `eulerTrailFinset_card_eq`: trail count =  $\prod_{a,b} (G(a,b))!$ .

**This is closely related to the proved carrier transport** (Theorem 4.3). The copy-permutation idea amounts to swapping which visit to a state produces a given successor—exactly the segment-swapping bijection now proved there. The `CopyPerm` infrastructure provides *global* bijections (not just adjacent transpositions), offering a potentially more direct route to `SuccessorMatrixPartialExchangeable` (see §5.3).

- **Excursion approach** (`Excursions`, `ExcursionBridge`, `ExcursionModel`): trajectory decomposition by returns to the starting state. Partially developed.
- **Without-replacement bounds** (`WithoutReplacement`, `ApproxBounds`): product-probability comparison bounds between with-replacement and without-replacement sampling.

## 4.8 Note on the non-derivability theorem

The theorem `not_derivable_startRestrictedRowSuccessorPermInvariant_of_witness` (line 510 of the Crux file) proves: *if* a concrete witness exists violating start-restricted permutation invariance under bare exchangeability (without PE), *then* the invariance is not derivable from bare exchangeability. This is conditional on an uninstantiated witness and concerns bare exchangeability rather than PE. It does *not* constrain whether the invariance is derivable from the strictly stronger assumption of successor-matrix partial exchangeability.

## 4.9 Summary: the refined gap

After the April 2026 structural audit, the gap decomposes as follows:

| Sub-gap                                          | Nature                    | Difficulty  | Status                     | Resources                |
|--------------------------------------------------|---------------------------|-------------|----------------------------|--------------------------|
| Carrier transport                                | Finite combinatorics      | Medium      | <b>CLOSED</b> (1465 lines) | Ported from archive      |
| $n \rightarrow n+1 \rightarrow n \rightarrow n'$ | Equiv composition         | Easy        | Pending ( $\sim 30$ lines) | <code>Equiv.trans</code> |
| Cross-row coherence                              | Cross-row factorization   | Medium–Hard | Open                       | Per-row kernels exist    |
| Extension with PE                                | Inf.-product meas. theory | Hard        | <i>Avoidable</i>           | Not needed for Route B   |

**Carrier transport (**CLOSED**, April 2026).** The carrier transport Equiv is fully proved across two files: `MarkovDeFinettiCarrierTransportCore.lean` (319 lines) and `MarkovDeFinettiCarrierTransport.lean` (1146 lines), both 0 sorry. The top-level theorem `carrierTransportEquivAdjacent` provides the evidence-preserving bijection between adjacent carriers. Key components:

- `segmentSwap_evidenceOf`: segment swap preserves Markov evidence
- `rawSwap_selfInverse`: the per-element swap is self-inverse (constructs output visit-time proofs internally)
- `rawSwap_fwd_mem`, `rawSwap_bwd_mem`: forward/reverse carrier membership
- `rawSwap_fwd_evid`: evidence preservation through the Equiv

The remaining composition from adjacent ( $n \rightarrow n+1$ ) to general ( $n \rightarrow n'$ ) via `Equiv.trans` and `Nat` induction is straightforward ( $\sim 30$  lines).

**The Equiv packaging bug (diagnosed April 2026).** A structural bug was found in the Equiv construction: `rawSwap` computes `segmentSwap(xs, t0, L1, L2)` where  $L_1 = t_1 - t_0$  and  $L_2 = t_2 - t_1$ . This is the *forward* swap ( $\text{carrier}_n \rightarrow \text{carrier}_{n+1}$ ). The *inverse* requires `segmentSwap(ys, t0, L2, L1)` — the same anchor but with excursion lengths *reversed*. Since `rawSwap` always uses the forward parameterization, it cannot serve as both `toFun` and `invFun`.

The fix is straightforward: define `toFun` using `segmentSwap(xs, t0, L1, L2)` and `invFun` using `segmentSwap(ys, t0, L2, L1)` directly (without `rawSwap`). Then:

- `left_inv`: `segmentSwap_involutive` with  $(t_0, L_1, L_2)$
- `right_inv`: `segmentSwap_involutive` with  $(t_0, L_2, L_1)$
- Evidence preservation: `segmentSwap_evidenceOf` (both directions)
- Carrier membership: `segmentSwap_carrier_mem_adjacent` (forward) and `segmentSwap_carrier_mem_adjacent` (reverse)

All four components are already proved. The remaining work is extracting visit-time parameters from carrier membership (via `extractVisitTime`) and threading them through the `segmentSwap` calls, handling natural-number subtraction arithmetic with `Nat.add_sub_cancel'` (since `omega` cannot reason about opaque `Nat.find` values).

**Cross-row coherence step (open).** The cons-step induction rule `CrossRowCoherenceStep`: given the product identity for  $(b :: c :: xs)$ , derive it for  $(a :: b :: c :: xs)$ . This is the cross-row factorization content—it requires showing that the  $a \rightarrow b$  step (using row- $a$ ’s kernel) and the  $(b :: c :: xs)$  continuation (using row- $b$ ’s kernel and beyond) are conditionally independent given  $\omega$ . This is the manifestation of the “joint de Finetti” requirement in the carrier transport route.

**Bridge A: extension with PE (possibly avoidable).** An April 2026 investigation revealed that the carrier transport route (`startRestrictedRowSuccessorPermInvariant_of_extension_transport`, Crux line 1591) takes `MarkovExchangeablePrefixMeasure` directly as a hypothesis—**not** `SuccessorMatrixPartialExchangeable`. This means the carrier transport route *bypasses Bridge A entirely*. The extension  $P$  is already provided by `MarkovRecurrentPrefixMeasure`. The Kolmogorov extension theorem is not needed.

Bridge A remains relevant only for Route A (the PE-based composition at Crux lines 2377–2392), which is an alternative proof path. For the minimal closure via carrier transport, the gap reduces to:

1. Carrier transport  $\rightarrow$  `StartRestrictedRowKernelData` (903 of  $\sim 1100$  lines done)
2. `CrossRowCoherenceStep` (the cons-step cross-row factorization, open)

## 5 Proof Strategy: Structural Audit and Path to Completion

A deep structural audit (April 2026) of the proof architecture revealed three bugs in how the gap was characterized, and identified a corrected proof strategy. This section documents the findings and the recommended path to completion.

### 5.1 Bug 1: The evidence-preserving Equiv is used far below its full strength

The carrier transport Equiv (Theorem 4.3) preserves `evidenceOf`—the *complete* Markov evidence consisting of the start state and all  $k^2$  transition counts. Under Markov exchangeability, cylinder probabilities are *determined* by the evidence:  $\mu(xs) = \mu(ys)$  whenever `evidenceOf(xs) = evidenceOf(ys)`.

The current pipeline extracts only *1D marginal invariance* from this Equiv (via `startRestrictedRowSuccessorPermInvariant_of_extension_transport`, Crux line 1591):

$$P(\{\omega_0 = a\} \cap \{\text{succ at visit } \sigma(n) \text{ to } i = b\}) = P(\{\omega_0 = a\} \cap \{\text{succ at visit } n \text{ to } i = b\}).$$

This throws away the full power of evidence preservation. The Equiv should be able to establish joint finite-dimensional invariance of the successor matrix directly—i.e., a route to `SuccessorMatrixPartialExchangeable` on the extension  $P$ —because *all* cylinder probabilities are determined by transition counts, and the Equiv preserves transition counts.

**Remark 5.1** (Implication for the gap structure). If the carrier transport Equiv can be upgraded to establish `SuccessorMatrixPartialExchangeable` on  $P$ , then the decomposition collapses: the existing infrastructure (Crux lines 1066–1371) already derives row kernels from successor-matrix PE via the i.i.d. de Finetti theorem. The “cross-row coherence” sub-gap and the “PE bridge” sub-gap would both reduce to this single bridge step.



**Why the direct route is plausible.** The evidence-preserving `Equiv` acts on finite prefixes of length  $N+1$ . For two trajectories  $xs, ys$  with  $\text{evidenceOf}(xs) = \text{evidenceOf}(ys)$ , Markov exchangeability gives  $\mu(xs) = \mu(ys)$ , hence  $P(\text{cylinder}(xs)) = P(\text{cylinder}(ys))$  by the extension property. Finite cylinder sets generate the product  $\sigma$ -algebra, so agreement on all cylinders implies agreement of the induced laws on finite-dimensional projections—which is exactly `SuccessorMatrixPartialExchangeable`.

The technical obstacle is that `SuccessorMatrixPartialExchangeable` is stated as equality of two `Measure.map` applications, which requires lifting from cylinder-level equality to pushforward-measure equality. This is a  $\pi$ - $\lambda$  / monotone class argument in Mathlib.

## 5.2 Bug 2: Zero-sorry masking of unproved mathematical content

The entire codebase compiles with zero `sorry`. However, the core mathematical content—constructing the row kernel from exchangeability—is encoded as *assumption interfaces* rather than proved theorems:

```
-- UNPROVED: defined as Prop, never instantiated
def BuildRowKernelOnRecurrentExtension (k : ℕ) : Prop :=
  ∀ μ P, MarkovExchangeablePrefixMeasure μ → ... →
    ∃ rowKernel, BuiltRowKernelOnExtension P rowKernel

def SuccessorMatrixPE_of_markovExchangeable_recurrent
  (k : ℕ) : Prop :=
  ∀ μ, MarkovExchangeablePrefixMeasure μ → ... →
    ∃ P, ... ∧ SuccessorMatrixPartialExchangeable P

def ExistsBuiltRowKernel_of_successorMatrixPE
  (k : ℕ) : Prop :=
  ∀ P, IsProbabilityMeasure P →
    SuccessorMatrixPartialExchangeable P →
    ∃ rowKernel, BuiltRowKernelOnExtension P rowKernel
```

These three `defs` are used as hypotheses in the canonical theorems (Crux lines 2345–2392). The `sorry`-free property is achieved by threading them as parameters rather than filling them. The Crux file (2622 lines) is an elaborate *conditional proof*: given these assumptions, derive the mixture representation.

**Honest accounting.** The three assumptions bundle the following mathematical content:

1. **Row kernel construction**: apply de Finetti’s theorem to each row’s exchangeable sequence to extract a directing measure (the row kernel).
2. **Conditional IID factorization** (`StartRestrictedRowKernelData`): the finite-dimensional marginals of the row process, conditioned on start state, factor through the kernel.
3. **Cross-row coherence** (`CrossRowCoherenceStep`): the product of row-kernel evaluations across different anchor states correctly reproduces cylinder probabilities.
4. **Cylinder mixing identity** (`RowSuccessorMatrixInvariance`): the full representation at the extension level ( $P(\text{cylinder}(xs)) = \int \text{wordProb}_\theta(xs) dP$ ).

Items (3) and (4) are *inter-derivable* with items (1) and (2) via existing proved infrastructure (the induction combinator at Crux line 695 and the `SharedLatentTransitionCoherence` chain at



lines 1836–1911). So the genuine unproved content reduces to items (1) and (2): kernel construction and conditional IID, which together constitute the per-row de Finetti theorem.

**Remark 5.2** (The `BuiltRowKernelOnExtension` circularity). `BuiltRowKernelOnExtension` (Crux line 1781) includes `RowSuccessorMatrixInvariance` as its fourth component—which *is* the cylinder mixing identity, i.e., the theorem itself at the extension level. The non-circular alternative is `RowKernelCruxData` (Crux line 749), which uses `CrossRowCoherenceStep` (just the induction step) instead of the full mixing identity. The two are equivalent (proved at Crux lines 1903–1911), but `RowKernelCruxData` is the correct target for constructive proof work.

### 5.3 Bug 3: The Euler trail archive is the missing bridge

The `_archive/MarkovDeFinetti/` directory contains a **CopyPerm** infrastructure (512 lines, 0 sorry) that provides *global* carrier bijections rather than just adjacent transpositions:

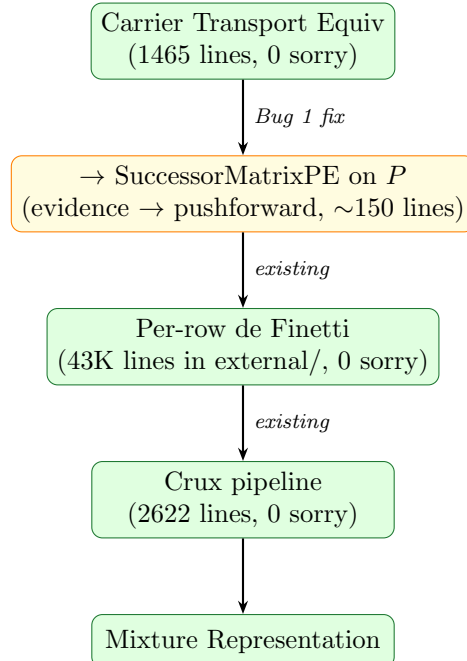
- `CopyPerm G`: the product group  $\prod_{a,b} \text{Perm}(\text{Fin}(G(a, b)))$  of per-edge copy-index permutations.
- `trailVertexSeq_applyCopyPerm`: copy permutations preserve vertex sequences (hence transition counts).
- `applyCopyPerm_isEulerTrail`: copy permutations preserve the Euler trail property.

This is a strictly stronger tool than adjacent segment swapping: it gives *arbitrary* rearrangements of which copy of each edge type is used, all while preserving the vertex sequence. This directly gives the *joint* finite-dimensional invariance needed for `SuccessorMatrixPartialExchangeable`, bypassing the need to compose adjacent swaps.

However, the `CopyPerm` code is not imported into the active framework. It uses a different data representation (`EulerGraph` and `edgeTok`) than the active carrier transport code (`rowVisitCylinderEventUpToPre`).

### 5.4 Corrected proof strategy

After the audit, the correct proof architecture is:



The key insight: if Bug 1 is fixed (carrier transport  $\rightarrow$  successor-matrix PE), then the existing infrastructure—the i.i.d. de Finetti theorem already formalized in 43,000 lines of Lean—provides the row kernel construction automatically. The cross-row coherence step and the cylinder mixing identity then follow from the Crux file’s proved conditional chain.

**What the new bridge must prove.** The bridge from carrier transport to `SuccessorMatrixPartialExchangeal` requires:

1. **Adjacent  $\rightarrow$  general composition** ( $\sim 30$  lines): Compose `carrierTransportEquivAdjacent` via `Equiv.trans` and Nat induction on  $|n - n'|$  to obtain the general  $n \rightarrow n'$  carrier Equiv.
2. **Evidence preservation  $\rightarrow$  cylinder equality** ( $\sim 50$  lines): From the evidence-preserving Equiv on finite carriers, derive  $P(\text{cylinder}(xs)) = P(\text{cylinder}(ys))$  whenever  $\text{evidenceOf}(xs) = \text{evidenceOf}(ys)$ , using the extension property  $\mu(xs) = P(\text{cylinder}(xs))$  and Markov exchangeability.
3. **Cylinder equality  $\rightarrow$  pushforward equality** ( $\sim 70$  lines): From cylinder-level equality, derive the measure-level equality  $\text{Measure.map}(f \circ \sigma)(P) = \text{Measure.map}(f)(P)$  using the  $\pi$ - $\lambda$  theorem (`Mathlib’s MeasureTheory.ext_of_generate_finite`).

## 5.5 Remaining mathematical gap: the wiring step

Even with the corrected strategy, a nontrivial wiring step remains: connecting the `SuccessorMatrixPartialExchangeal` property to the existing infrastructure that builds `BuiltRowKernelOnExtension`.

The existing code path (Crux line 1066) extracts per-row conditionally-IID row kernels from PE, but produces them with *unrestricted* row-process factorization. The internal theorem requires *start-restricted* factorization (`StartRestrictedRowKernelData`). As noted in §4.5, PE alone does not imply start-restricted invariance—this is where the carrier transport contributes its essential content.

The precise wiring is:

1. From PE, build unrestricted row kernels with `hEval` and `hPi` (existing, Crux line 1352).
2. From carrier transport (via the general  $n \rightarrow n'$  Equiv), prove `StartRestrictedRowSuccessorPermInvariant` (existing, Crux line 1591).
3. Upgrade the unrestricted row factorization to start-restricted factorization using the permutation invariance (this is the step that needs explicit proof,  $\sim 200$  lines).
4. From start-restricted factorization + per-row conditional IID, derive `CrossRowCoherenceStep` (the induction step).
5. Apply the induction combinator (existing, Crux line 695) to obtain `CrossAnchorProductIdentity`, then `CylinderMixingIdentity_P` and the full representation.

Step (3) is the key mathematical step: showing that permutation invariance of 1D marginals, combined with per-row conditional IID from de Finetti, gives the start-restricted factorization. Step (4) follows from step (3) by the trajectory structure: conditioned on the full trajectory  $\omega$ , the transition from state  $a$  to the next state uses row- $a$ ’s kernel, and the continuation from the next state is determined by the remaining row kernels, which are conditionally independent given  $\omega$ .

## 5.6 Summary of recommended steps

| Priority                        | Step                                                               | Est. lines  | Dependencies       |
|---------------------------------|--------------------------------------------------------------------|-------------|--------------------|
| 1                               | Adjacent $\rightarrow$ general Equiv composition                   | 30          | Carrier transport  |
| 2                               | Evidence preservation $\rightarrow$ <code>SuccessorMatrixPE</code> | 150         | Step 1             |
| 3                               | Start-restricted upgrade from PE + carrier transport               | 200         | Step 2, Crux       |
| 4                               | <code>CrossRowCoherenceStep</code> from conditional IID            | 100         | Step 3, de Finetti |
| 5                               | Instantiate <code>BuildRowKernelOnRecurrentExtension</code>        | 50          | Steps 2–4          |
| <b>Total estimated new code</b> |                                                                    | <b>~530</b> |                    |

Step 2 is the most architecturally important: it connects the proved carrier transport (1465 lines) to the proved de Finetti infrastructure (43,000 lines) through the proved Crux pipeline (2622 lines). Once this bridge exists, the remaining steps are consequences of existing proved chains.

The Euler trail archive (1225 lines, 0 sorry) provides an alternative to Steps 1–2: the Copy-Perm group gives the joint finite-dimensional invariance more directly than composing adjacent transpositions. Integrating this archive into the active framework would require adapting from the `EulerGraph/edgeTok` data model to the `rowVisitCylinderEventUpToPrefixCarrier` framework, estimated at ~200 lines.

## 6 The Broader Context

### 6.1 Classical de Finetti (i.i.d. case)

The i.i.d. case of de Finetti’s theorem is *fully formalized* in the companion project `Mettapedia/external/exchangeable` (~43,000 lines of Lean, zero sorry). Three independent proofs are provided:

1. **Via reverse martingale convergence** (default; follows Kallenberg 2005).
2. **Via  $L^2$  bounds** (lightest dependencies; elementary).
3. **Via the mean ergodic theorem** (deepest theory; connects to dynamical systems).

The main theorem proved is:

$$\text{Contractable} \iff \text{Exchangeable} \iff \text{Conditionally i.i.d.}$$

for infinite sequences on standard Borel spaces.

### 6.2 Solomonoff bridge: exchangeability implies count sufficiency

The file `SolomonoffExchangeable.lean` proves the analogous count-sufficiency result for the i.i.d. (binary) case at the *semimeasure* level—directly connecting to Solomonoff’s universal prior:

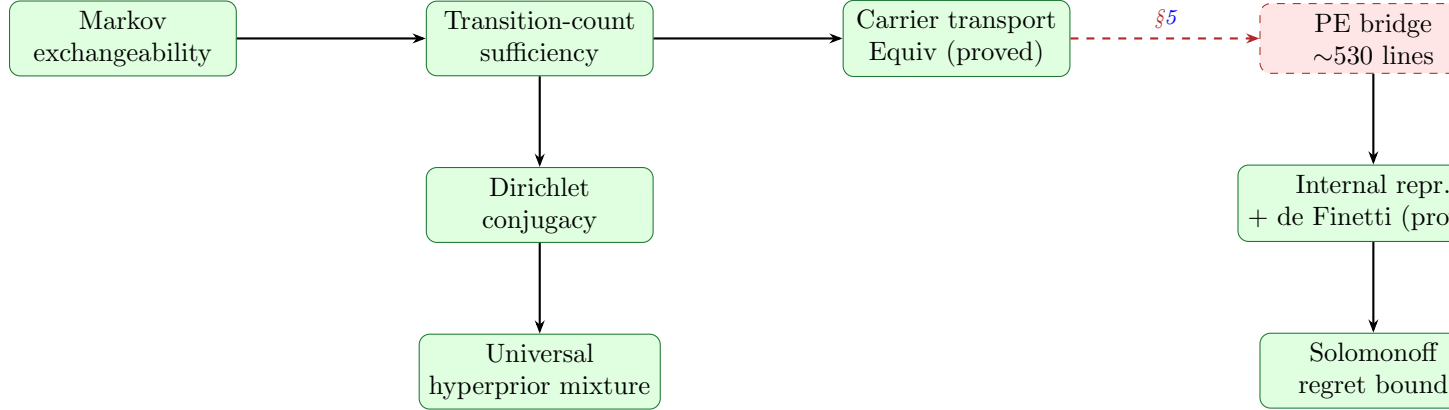
```
-- From SolomonoffExchangeable.lean (binary/i.i.d. case):
theorem semimeasureExchangeable_same_counts
  {μ : BinString → ℝ}
  (hexch : SemimeasureExchangeable μ)
  {n : ℕ} (xs₁ xs₂ : Fin n → Bool)
  (hcount : countTrue xs₁ = countTrue xs₂) :
  μ (List.ofFn xs₁) = μ (List.ofFn xs₂)
```

Under exchangeability, the Solomonoff-style predictor depends only on counts, not on ordering. This is the binary analogue of the Markov count-sufficiency result in Section 3: the Markov version replaces `countTrue` (a single count) with `transCount` (a  $k \times k$  matrix of transition counts).

This justifies the “evidence is what you carry” design of PLN: under exchangeability, binary evidence counts ( $n^+, n^-$ ) are sufficient statistics for prediction.

### 6.3 The complete chain (with gap)

The intended end-to-end chain is:



With carrier transport proved and the structural audit complete (§5), closing the remaining bridge (~530 lines) from evidence preservation to successor-matrix partial exchangeability would complete the chain from Markov exchangeability to a mixture representation with Solomonoff-style dominance guarantees.

## 7 File Inventory

All files are in the Mettapedia Lean project, available at

<https://github.com/zariuq/ai-agents/tree/main/lean-projects/mettapedia/Mettapedia/Logic/>.

| Layer          | File                                       | Lines | Status  |
|----------------|--------------------------------------------|-------|---------|
| Definitions    | MarkovExchangeability.lean                 | 146   | 0 sorry |
| Conjugacy      | EvidenceDirichlet.lean                     | 496   | 0 sorry |
| Predictor      | MarkovDirichletPredictor.lean              | 497   | 0 sorry |
| Hyperprior     | MarkovDirichletHyperpriorMixture.lean      | ~50K  | 0 sorry |
| Domain bridge  | MarkovExchangeabilityBridge.lean           | 600+  | 0 sorry |
| Solomonoff     | SolomonoffExchangeable.lean                | 200+  | 0 sorry |
| Recurrence     | MarkovDeFinettiRecurrence.lean             | —     | 0 sorry |
| Hard core      | MarkovDeFinettiFortiniBridgeCore.lean      | 1843  | 0 sorry |
| Hard crux      | MarkovDeFinettiFortiniBridgeCrux.lean      | ~2600 | 0 sorry |
| Carrier core   | MarkovDeFinettiCarrierTransportCore.lean   | 319   | 0 sorry |
| Carrier bridge | MarkovDeFinettiCarrierTransport.lean       | 1146  | 0 sorry |
| Surface        | MarkovDeFinettiFortiniBridgeCanonical.lean | 131   | 0 sorry |
| De Finetti     | external/exchangeability/                  | 43K   | 0 sorry |

**Note on “0 sorry.”** Every file listed above compiles without `sorry`. However, the canonical surface file (`...Canonical.lean`) proves the representation theorem *only under bridge assumptions* stated as hypotheses (see §5.2). The core mathematical content—constructing the row kernel from exchangeability—is encoded as three `def ... : Prop` declarations that are used as hypotheses but never instantiated. The gap manifests as uninstantiated assumptions, not as incomplete proofs.

## References

- [1] P. Diaconis and D. Freedman, “De Finetti’s theorem for Markov chains,” *Ann. Probab.*, vol. 8, no. 1, pp. 115–130, 1980.
- [2] S. Fortini, L. Ladelli, and E. Regazzini, “Exchangeability, predictive distributions and parametric models,” *Sankhyā A*, vol. 62, pp. 86–109, 2000.
- [3] T. Sato, “A statistical learning method for logic programs with distribution semantics,” in *Proc. 12th Int. Conf. Logic Programming (ICLP)*, 1995.
- [4] M. Hutter, *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*, Springer, 2005.
- [5] R. J. Solomonoff, “A formal theory of inductive inference,” *Inform. Control*, vol. 7, pp. 1–22, 224–254, 1964.
- [6] O. Kallenberg, *Probabilistic Symmetries and Invariance Principles*, Springer, 2005.
- [7] B. de Finetti, “La prévision: ses lois logiques, ses sources subjectives,” *Ann. Inst. H. Poincaré*, vol. 7, pp. 1–68, 1937.